```
    this.flipState()
  }

  flipState(): void {
    this.statusValue = !this.statusValue
  }

  statusValueChanged(): void {
    this.updateCssClass()
  }

  updateCssClass(): void {
    for (const oneCssClass of this.cssClasses) {
      this.elementToChangeTarget.classList.toggle(
        oneCssClass,
        this.statusValue
      )
    }
  }
}
```

We are specifying that this class has a property `cssClass` that is a string, an `elementToChangeTarget` that is an `HTMLElement`, and a `statusValue` that is a `boolean`. TypeScript uses this information in a couple of ways.

The name `CssController` can be used as a type just like the basic types we have already covered. You can annotate a variable, function parameter, or return value to be of type `CssController` and the compiler will require that those instances use only properties that have been explicitly defined by the `CssController` class.

TypeScript expands upon JavaScript class syntax in that it allows you to avoid repeating a declaration of a property, the naming of the property in the list of constructor parameters, and the assigning of the property.

So instead of:

```
class User {
  firstName: string
  lastName: string

  constructor(firstName, lastName) {
```